

COMPSCI 701:
Creating Maintainable Software
Lecture 11.1: Improving Testability

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Agenda

- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

- Admin
 - SET starts
 - Test 2
 - Schedule Week 11
 - Monday — Testability
 - Tuesday — Test-driven Design & encapsulation case studies
 - Wednesday — class exercise
 - Schedule Week 12 (tentative)
 - Monday — “Measuring” maintainability
 - Tuesday — Revision
 - Wednesday — no lecture
 - Thursday 1800–2000 — Test 2
- Design choices and testability
- Use of Dependency Injection to improve testability

Test 2

- Agenda
- **Test 2**
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

- When: Thursday 23 October 1800-2000
- Locations: 401-401, Eng1401 Lecture Theatre. Family names Abao - Pham
401-439, Eng1439 Lecture Theatre. Family names Puarat - Zhu
- Duration: The test is designed to be 1 hour long. The 2 hour period is to allow for setting up and to provide flexibility
- Format: Short answers (possibly including MCQ)
- Content: **Everything** in the course presented up to and including Thursday 23 October
- Support: Restricted Book (Learning Journal)
- 2024 test available on Canvas

Previously in COMPSCI701

- Agenda
- Test 2
- **Review**
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

- **Testability** — degree of effectiveness and efficiency with which test criteria can be established for a system, product or component and tests can be performed to determine whether those criteria have been met.

- Agenda
- Test 2
- **Review**
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

Previously in COMPSCI701

- in order to test something, we need to be able to exercise it how we want so that we can observe the consequences of **different inputs** (controlling the state) ⇒ need good **controllability**
- in order to test something, we need to be able to see what it's **actually doing** (observing the state) ⇒ need good **observability**
- ⇒ to increase testability, increase controllability and observability

Previously in COMPSCI701

- Agenda
- Test 2
- **Review**
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

- ...
- Executing a test case requires
 - get the IUT into the pretest state
 - supply inputs
 - execute IUT with inputs
 - compare actual behaviour of IUT with expected behaviour

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

Can Design affect Testability?

```
public class Player {  
    private String _name;  
    private Board _board;  
    ...  
    public Player(Board board) { _board = board; }  
    ...  
}  
  
public class Board {  
    private List<House> _houses = ...  
    ...  
}
```

- In order to test Player, need to create a Player object.
- In order to create a Player object, need to create a Board object.
- To create a Board, need to create one or more Houses.
- Remove the dependency of Player on Board, and creating a Player object becomes much easier $\Rightarrow$ more testable

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

Example 1: Supermarket simulation

```
public class ManageSimulation {  
    private Simulation _sim;  
    public void run() {  
        while (true)  
            _sim.shopperArrives();  
        ....  
    }  
}
```

- Discrete-Event Simulation
 - events take place at discrete times
 - consequences of events alter the state of the system, including creating new events to take place in the future
 - how events are created often have a **random** element to their creation
- How to test that a given method in the Simulator is correct when its behaviour is random?

Example 1 continued

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

```
import java.util.Random;
public class Simulation {
    private Random _random = new Random();
    private List<Shopper> _currentShoppers;
    public void shopperArrives() {
        double r = _random.nextDouble();
        Shopper s = null;
        if (r < 0.33) {
            s = new ChaoticShopper();
        } else if (r < 0.66) {
            s = new OrganisedShopper();
        } else {
            s = new ImpatientShopper();
        }
        _currentShoppers.add(s);
    }
}
```

- Every time `shopperArrives()` is called there is likely to be **different behaviour**
- Very difficult to create **repeatable** tests — **lack of control**
-

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

Example 2: Testing a Book class

```
public class Book {  
    private String _barcode;  
    private Biblio _biblio;  
    private String _format;  
    private ItemStatus _status;  
    private InventoryLifecycle _lifecycle;  
    private Date _purchaseDate;  
    private Money _fee;  
    private String _notes;  
    ...  
}
```

- Many classes have many fields and methods
- $\Rightarrow$ getting them into the right pre-test state can require complex test code that is difficult to get right — control again

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

Example 3: Testing classes with complex components

```
public interface DBConnection {
    public void open();
    public void close();
    public ResultSet query(String query);
    ...
}
public class Library {
    DBConnection _db; // connects to external database
    ....
    public Library() {
        _db = new DBConnection(...);
    }
    public List<Book> getOverDue() {
        //External database operations can be slow
        //Also need that database to be set up correctly for this test
        //(every time the test is run...)
        ResultSet rs = _db.query("SELECT * WHERE duedate < " +
            Calender.today());
        ...
    }
    ...
}
```

- Some classes depend on other (perhaps even more) complex classes
- These complex classes may create performance problems for tests

- Agenda
- Test 2
- Review
- Motivation
- **Testability**
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

Using DIP to improve testability

- Need to increase observability and controllability
 - Avoid breaking encapsulation if possible (or reduce the effect as much as possible)
 - Observation: the more concrete the state, the harder it is to control, so how to make state less concrete?
- ⇒ Dependency Inversion Principle
- In particular, **dependency injection** gives us control as to what the state is

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

Example 1: Apply DIP

```
public class Simulation {  
    private Random _random;  
    public void setRandom(Random random) {  
        _random = random;  
    }  
    ....remaining fields and methods as before  
}
```

- Using **setter** injection
- Strictly-speaking not DIP because Random is concrete

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

Example 1: Testing (the wrong way)

```
// Check the behaviour simulator when a ChaoticShopper arrives
public class TestSimulation extends TestCase {
    public void testChaotic() {
        Simulation sim = new Simulation();
        // Specify the random distribution for test
        sim.setRandom(new Random());
        ...
        sim.shopperArrives();
        // can't guarantee that the new shopper is in fact a
        // ChaoticShopper() so don't know what to test
        ...
    }
}
```

- Changed the test because the design changed, but haven't solved the problem

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- **Mock Testing**
- Mock Variants
- Maintainability
- Key Points

Example 1: Use of “Mocking”

```
// Not the real ChaoticRandomGen but a mockup of one
public class MockChaoticRandomGen extends Random {
    public double nextDouble() {
        return 0.30;
    }
}

public class TestSimulation extends TestCase {
    public void testChaotic() {
        Simulation sim = new Simulation();
        // Specify the random distribution for test
        Random distribution = new MockChaoticRandomGen() // control
        sim.setRandom(distribution); //Injection site
        sim.shopperArrives();
        // Now guaranteed that new shopper is chaotic

        // other parts of test
    }
}
```

- When we create “mock” objects, we can have them behave in a way that the real object would behave, but with complete **control** over what actually happens.

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- **Mock Testing**
- Mock Variants
- Maintainability
- Key Points

Example 1: Use of “Mocking”

```
// Not the real ChaoticRandomGen but a mockup of one
public class MockChaoticRandomGen extends Random {
    private double _result;
    private int _count;
    public ChaoticRandomGen(double r) {
        _result = r;
    }
    public double nextDouble() {
        _count++;
        return _result;
    }
    public void assertTimesCalled(int expected) {
        assertEquals(expected, _count);
    }
}

public class TestSimulation extends TestCase {
    public void testChaotic() {
        Simulation sim = new Simulation();
        // Specify the random distribution for test
        Random distribution = new MockChaoticRandomGen() // control
        sim.setRandom(distribution); //Injection site
        sim.shopperArrives();
        // Now guaranteed that new shopper is chaotic
        distribution.assertTimesCalled(1); // observe
        // other parts of test
    }
}
```

- When we create “mock” objects, we can have them behave in a way that the real object would behave, but with complete **control** over what actually happens.
- And once we start creating mock classes, we might as well make them as useful as possible by adding more responsibilities that support the testing process, including providing the means to **observe** what happens.

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- **Mock Testing**
- Mock Variants
- Maintainability
- Key Points

Other consequences

```
public class Simulation {
    private Random _random;
    public void setRandom(Random random) {
        _random = random;
    }
    ....remaining fields and methods as before
}
public class ManageSimulation {
    private Simulation _sim;
    public void run() {
        // Decide distribution for random events
        if (_currentTime == Night) {
            _sim.setRandom(new NightRandomGen()); //Injection site
        } else {
            _sim.setRandom(new DayRandomGen());    //Injection site
        }
        // Main simulation loop
        while (true) {
            ...
        }
    }
}
```

- Had we gone with DIP at the beginning (rather than as a way to improve testability), we would have had other benefits, e.g changing distribution used at run-time

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- **Mock Testing**
- Mock Variants
- Maintainability
- Key Points

Example 2 — Using Mocks

```
public class Book {
    private String _barcode;
    ...
    public Book(String barcode, Biblio biblio,
                String format, ItemStatus status,
                ...) {
        _barcode = barcode;
        _biblio = biblio;
        ...
    }
}

public class TestItem extends TestCase {
    public void testOne() {
        Book book = new Book("xkcd208", new MockBiblio(),
                            "paperback", new MockStatus(),
                            ...);
        ...
    }
}
```

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- **Mock Testing**
- Mock Variants
- Maintainability
- Key Points

Example 3 — Using Mocks

```
public interface DBConnection {
    public void open();
    public void close();
    public ResultSet query(String query);
    ...
}

public class MockSELECTDB implements DBConnection {
    private boolean _open = false;
    public void open() {
        _open = true;
    }
    public void close() {
        _open = false;
    }
    public ResultSet query(String query) {
        if (!query.startsWith("SELECT")) {
            throw new RuntimeException("Unexpected query: "+query);
        }
        ResultSet rs = new ResultSet();
        rs.add(value1);
        rs.add(value2);
        ...
    }
}
```

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- **Mock Variants**
- Maintainability
- Key Points

Mock Variants

- Stubs
 - completely trivial implementations, provided as placeholders to allow things to compile and be created
 - commonly talked about in **intergration testing**
 - Usual form requires changing the code being tested
- Fakes — very simple mocks
- Mocks — as well as supplying controlled behaviour, can observe how they are being used
 - <http://jmock.org>
 - <http://easymock.org>
 - <http://mockito.org>
- Used In Real Life — GoogleMock (part of <https://github.com/google/googletest>)

Creating Maintainable Software

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

- Creating software involves many decisions
- It would be nice to know the right decision to make all the time
- Some decisions can be guided by heuristics, principles, and advice
- But some decisions are difficult to make even with such guidance
- What is needed is a **process** that usually (or at least often) helps make a good decision
- One candidate for such a process is **test-driven development**
 - development is guided by **already-written** tests
 - tests must be able to be executed $\Rightarrow$ design must support testing
 - So, design must be **testable**

- Agenda
- Test 2
- Review
- Motivation
- Testability
- Applying DI
- Mock Testing
- Mock Variants
- Maintainability
- Key Points

Key Points

- Choice of design can affect testability
- In particular use of DIP (dependency injection) often improves testability (but there are other reasons to use this)